

LLM TABANLI YAZILIM HATA ANALİZİ VE KARŞILAŞTIRMA PLATFORMU

Öğrenciler: Damla Nur Alper & Sidal Deniz Bingöl

Tarih: 23 Aralık 2025

Ders: Yapay Zeka Destekli Yazılım Geliştirme

İÇİNDEKİLER

- Proje Özeti
- Giriş ve Problem Tanımı
- Metodoloji
- Teknik Mimari
- Uygulama Süreci
- Sonuçlar ve Bulgular
- Karşılaşılan Zorluklar
- Sonuç ve Değerlendirme
- Kaynaklar

1. PROJE ÖZETİ

Bu projede, **6 farklı Large Language Model (LLM)**'in yazılım hatalarını analiz etme performansını karşılaştıran **akademik bir benchmark platformu** geliştirilmiştir. Platform, **100 farklı hata kategorisinden** oluşan bir veri seti üzerinde LLM'leri **6 farklı değerlendirme kriteri** kullanarak analiz etmektedir.

Değerlendirme kriterleri aşağıdaki gibidir:

- Teknik Doğruluk
- Çözüm Kalitesi
- Açıklık
- Özlülük
- Hız
- Güvenilirlik

Temel Bulgular

- En İyi LLM:** Cohere (84.26 / 100)
- En Zayıf LLM:** OpenRouter Hermes (14.68 / 100)
- Toplam Veri:** 301 kayıt, 2 geliştirici, 6 LLM

- Değerlendirme Yöntemi: Ağırlıklı puanlama sistemi

2. GİRİŞ VE PROBLEM TANIMI

2.1. Problem

Yazılım geliştirme süreçlerinde karşılaşılan hatalar, geliştiriciler için ciddi zaman kaybına neden olmaktadır. Son yıllarda **Large Language Model (LLM)** tabanlı sistemler yazılım hatalarının tespiti ve çözümünde aktif olarak kullanılmaya başlanmıştır. Ancak mevcut durumda, hangi LLM'in yazılım hata analizi konusunda daha başarılı olduğuna dair **bilimsel ve ölçülebilir bir karşılaştırma** bulunmamaktadır.

2.2. Motivasyon

Bu projenin gerçekleştirilmesindeki temel motivasyonlar şunlardır:

- LLM'lerin yazılım hata analizi kabiliyetlerini **objektif kriterler** ile karşılaştırmak
- Geliştiricilere, **hangi LLM'in tercih edilmesi gerektiği** konusunda veri sunmak
- Ücretsiz ve ücretli API'ler** arasındaki performans farklarını gözlemlemek

2.3. Proje Hedefleri

- 100 farklı yazılım hata türünü** içeren kapsamlı bir veri seti oluşturmak
- 6 farklı LLM'den** yanıt toplamak (Groq, Mistral, Cohere, OpenRouter x3)
- Akademik kriterlere dayalı bir **değerlendirme algoritması** geliştirmek
- İstatistiksel olarak **anlamlı sonuçlar** elde etmek

3. METODOLOJİ

3.1. Kullanılan LLM'ler

Aşağıda projede kullanılan LLM'ler ve temel özellikleri yer almaktadır:

LLM	Model	API	Özellik
Groq	Llama 3.3 70B	Ücretsiz	Hızlı inference, cömert limitler

Mistral	Mistral Small (~22B)	Resmi API	Yüksek kalite
Cohere	Command-R	Enterprise	Güçlü mantık yürütme
OpenRouter Llama	Llama 3.2 3B Free	Ücretsiz	Günlük kota sınırlı
OpenRouter Mistral	7B Free	Ücretsiz	Günlük kota sınırlı
OpenRouter Hermes	405B Free	Ücretsiz	Günlük kota sınırlı

Not: Gemini API anahtarı kullanılmadığı için değerlendirmeye dahil edilmemiştir.

3.2. Veri Toplama Süreci

Hata Kategorileri (Ana Kategoriler)

Projede, 17 Aralık dersi kapsamında belirlenen hata kategorileri kullanılmıştır:

- API Hataları (API_ERR)
- Otomasyon Hataları (AUTO_ERR)
- Tarayıcı Hataları (BROWSER_ERR)
- Kodlama Hataları (CODE_ERR)
- Yapılandırma Hataları (CONFIG_ERR)
- Veri Hataları (DATA_ERR)
- Veritabanı Hataları (DB_ERR)
- Çevresel Hatalar (ENV_ERR)
- Ağ Hataları (NET_ERR)
- Performans Hataları (PERF_ERR)
- Güvenlik Hataları (SEC_ERR)
- Sürüm Uyumsuzluğu Hataları (VERSION_ERR)

Veri Üretim Stratejisi

Veri üretimi sürecinde çeşitliliği ve tarafsızlığı sağlamak amacıyla aşağıdaki stratejiler uygulanmıştır:

- **SEED Sistemi:**
Her geliştirici farklı bir SEED değeri kullanmıştır.
 - Damla: 500
 - Sidal: 1000
- **Rastgele Seçim:**
Her geliştirici, 100 hata türü içerisinde rastgele seçim yapmıştır.
- **Farklı Prompt'lar:**
Aynı hata türü için dahi farklı prompt metinleri oluşturulmuştur.

Sonuç:

- Damla: 201 kayıt
- Sidal: 100 kayıt
- Toplam:** 301 kayıt

3.3. Değerlendirme Kriterleri

Platform, her bir LLM yanıtını **6 farklı kriter** kullanarak değerlendirmektedir.

Değerlendirme Kriterleri Tablosu

Kriter	Ağırlık	Açıklama
Technical Accuracy	%25	Hatanın doğru teşhis edilmesi
Solution Quality	%25	Önerilen çözümün etkinliği
Clarity	%20	Açık ve anlaşılır açıklama
Conciseness	%10	Gereksiz detaylardan kaçınma
Speed	%10	Yanıt süresi performansı
Reliability	%10	Tutarlılık ve hatasız çalışma

Puanlama Yöntemi

Değerlendirme sürecinde aşağıdaki yöntemler kullanılmıştır:

- Her kriter için **NLP tabanlı feature extraction**
- Bag-of-Words** modeli ile anahtar kelime tespiti
- Response time** metriği ile hız skoru hesaplama

- **API error** kontrolü ile güvenilirlik skoru belirleme

Toplam Skor:

Her LLM için **0–100** arası ağırlıklı puan hesaplanmıştır.

```
for llm_name, responses in llm_responses.items():
    total_score = 0.0
    valid_count = 0
    criterion_totals = {
        'technical_accuracy': 0,
        'solution_quality': 0,
        'clarity': 0,
        'conciseness': 0,
        'speed': 0,
        'reliability': 0
    }

    for response_obj in responses:
        features = self.extractor.extract(response_obj['text'])

        scores = self.scorer.score_response(
            features,
            response_obj['response_time'],
            response_obj['is_error'],
            WEIGHTS
        )

        total_score += scores['total']
        valid_count += 1

        for criterion in criterion_totals.keys():
            criterion_totals[criterion] += scores[criterion]
```

4. TEKNİK MİMARİ

4.1. Teknoloji Yığını

Backend

- **Node.js (v18+)**: Veri üretimi ve API entegrasyonu
- **Python 3.11**: Evaluation algoritması
- **PostgreSQL**: Veritabanı yönetimi

Kullanılan Kütüphaneler

- **axios**: HTTP istekleri
 - **dotenv**: Çevre değişkenleri yönetimi
 - **pg**: PostgreSQL client (Node.js)
 - **psycpg2-binary**: Python PostgreSQL driver
-

4.2. Database Schema

Tablo Adı: `llm_error_analysis`

Toplam Kolon Sayısı: 25

Temel Kolonlar

- `id` (SERIAL): Birincil anahtar
- `developer_name` (TEXT): Veri üreticisi
- `created_at` (TIMESTAMP): Oluşturulma zamanı
- `error_category` (TEXT): Hata kategorisi
- `error_code` (TEXT): Hata kodu
- `error_message` (TEXT): Hata mesajı
- `prompt_sent` (TEXT): LLM'e gönderilen prompt

LLM Response Kolonları (6 LLM × 2 = 12 kolon)

- `groq_response`, `groq_response_time`
- `mistral_response`, `mistral_response_time`
- `cohere_response`, `cohere_response_time`
- `openrouter_response`, `openrouter_response_time`
- `openrouter_mistral_response`, `openrouter_mistral_response_time`
- `openrouter_hermes_response`, `openrouter_hermes_response_time`

Evaluation Sonuç Kolonları

- `best_llm` (TEXT): En iyi performans gösteren LLM
- `worst_llm` (TEXT): En düşük performans gösteren LLM
- `description` (TEXT): Detaylı karşılaştırma raporu (~4200 karakter)

llm_error_analysis

Columns (25)

id

developer_name

created_at

error_category

error_code

error_message

prompt_sent

gemini_response

groq_response

mistral_response

best_llm

notes

cohere_response

openrouter_response

gemini_response_time

groq_response_time

mistral_response_time

cohere_response_time

openrouter_response_time

openrouter_hermes_response

openrouter_hermes_response_time

openrouter_mistral_response

openrouter_mistral_response_time

worst_llm

description

	id [PK] integer	developer_name text	created_at timestamp without time zone	error_category text	error_code text	description text	best_llm text	worst_llm text
1	3	Developer	2025-12-21 22:45:49.636263	SEC_ERR	TokenInvalid	Token formatı geçersiz veya imza doğrulanamadı	cohere	openrouter_hermes
2	7	Developer	2025-12-21 23:13:23.107113	AUTO_ERR	ElementNotInteractableException	Element görünür ancak etkileşime kapalı durumda	cohere	openrouter_hermes
3	17	Developer	2025-12-22 14:47:38.503187	DB_ERR	SQLIntegrityConstraintViolationE...	Foreign key veya unique constraint ihlali	cohere	openrouter_hermes
4	20	Developer	2025-12-22 14:49:41.756737	BROWSER_ERR	WebGLNotSupported	Tarayıcı WebGL desteklemiyor veya devre dışı	cohere	openrouter_hermes
5	21	Developer	2025-12-22 14:50:39.98318	DATA_ERR	ConstraintViolationOnInsert	Veritabanı kısıtlaması ihlal edildi, unique constrai...	cohere	openrouter_hermes
6	29	Developer	2025-12-22 15:03:58.453893	API_ERR	400	İstek formatı hatalı veya eksik parametreler mevc...	cohere	openrouter_hermes
7	58	Damla	2025-12-22 15:18:21.063691	CODE_ERR	NullPointerException	Null referansa erişim denemesi, nesne başlatılma...	cohere	openrouter_hermes
8	44	Developer	2025-12-22 15:09:54.101072	CODE_ERR	ArrayIndexOutOfBoundsException	Dizi sınırları dışında index erişimi	cohere	openrouter_hermes
9	50	Developer	2025-12-22 15:11:55.669025	PERF_ERR	HighMemoryUsage	Bellek kullanımı kritik seviyede	cohere	openrouter_hermes
10	59	Damla	2025-12-22 15:18:36.36001	CODE_ERR	ClassCastException	Geçersiz tip dönüşümü, uyumsuz sınıflar arası ca...	cohere	openrouter_hermes
11	68	Damla	2025-12-22 15:22:31.816637	PERF_ERR	DatabaseTimeout	Veritabanı sorgusu çok uzun sürdü	cohere	openrouter_hermes
12	69	Damla	2025-12-22 15:22:50.800763	PERF_ERR	APIResponseDelay	API yanıt süresi SLA'yı aşıyor	cohere	openrouter_hermes

4.3. API Konfigürasyonu

Çevre Değişkenleri (.env)

Aşağıda projede kullanılan çevre değişkenleri yer almaktadır:

Database

DB_HOST=localhost

DB_PORT=5432

DB_NAME=llm_error_analysis

DB_USER=postgres

DB_PASSWORD=****

API Keys

GROQ_API_KEY=****

MISTRAL_API_KEY=****

COHERE_API_KEY=****

OPENROUTER_API_KEY=****

GEMINI_API_KEY=****

Config

DEVELOPER_NAME=Damla

SEED=500

5. UYGULAMA SÜRECİ

5.1. Database Kurulumu

Adımlar

- PostgreSQL kurulumu ve başlatılması
- `src/database/schema.sql` dosyası ile tablo oluşturulması
- `scripts/setup/create-db-now.js` script'i ile database'in hazırlanması

```
CREATE TABLE llm_error_analysis (  
  id SERIAL PRIMARY KEY,  
  developer_name TEXT NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  error_category TEXT NOT NULL,  
  error_code TEXT NOT NULL,  
  error_message TEXT NOT NULL,  
  prompt_sent TEXT NOT NULL,  
  gemini_response TEXT,  
  groq_response TEXT,  
  mistral_response TEXT,  
  best_llm TEXT,  
  notes TEXT
```

```
ALTER TABLE llm_error_analysis
ADD CONSTRAINT check_error_category
CHECK (error_category IN (
    'API_ERR', 'AUTO_ERR', 'BROWSER_ERR', 'CODE_ERR',
    'CONFIG_ERR', 'DATA_ERR', 'DB_ERR', 'ENV_ERR',
    'NET_ERR', 'PERF_ERR', 'SEC_ERR', 'VERSION_ERR'
```

5.2. Veri Üretimi

Script: `src/generate-3-llm.js`

İşleyiş

- `.env` dosyasından SEED ve DEVELOPER_NAME bilgileri okunur
- SEED değeri kullanılarak rastgele 100 hata türü seçilir
- Her hata için detaylı bir prompt oluşturulur
- 3 LLM'e paralel istek atılır (Groq, Mistral, Cohere)
- Yanıtlar ve response time bilgileri kaydedilir
- Rate limit durumunda 3 saniye bekleme uygulanır

Prompt

Aşağıdaki yazılım hatasını analiz edin ve detaylı bir çözüm önerisi sunun:

Hata Kategorisi: Güvenlik Hataları (SEC_ERR)

Hata Kodu: TokenInvalid

Hata Adı: TokenInvalid

Hata Mesajı: Token formatı geçersiz veya imza doğrulanamadı

Lütfen aşağıdaki başlıklar altında analiz yapın:

1. Hatanın Nedeni: Bu hatanın oluşma sebepleri nelerdir?
2. Hata Senaryosu: Hangi durumlarda bu hata ile karşılaşılır?
3. Çözüm Önerileri: Bu hatayı nasıl çözebiliriz? (Adım adım)
4. Önleme Yöntemleri: Gelecekte bu hatanın oluşmasını nasıl engelleriz?
5. Best Practices: İlgili konuda en iyi uygulamalar nelerdir?

✕ Cancel

✓ OK

ÖRNEK BİR LLM CEVABI (GROQ):

1. Hatanın Nedeni:

ArrayIndexOutOfBoundsException, bir diziye erişirken belirtilen indexin dizi sınırları dışında olması durumunda ortaya çıkan bir hatadır. Bu hata genellikle aşağıdaki nedenlerle oluşabilir:

- Diziye erişim sırasında kullanılan index değerinin dizi boyutundan büyük veya eşit olması.
- Diziye erişim sırasında kullanılan index değerinin negatif olması.
- Dizi null veya boşken bir index'e erişilmek istenmesi.

2. Hata Senaryosu:

Bu hata ile aşağıdaki durumlarda karşılaşılabilir:

- Bir döngü içinde dizi elemanlarına erişilirken, döngü sınırı dizi boyutundan büyük belirlenirse.
- Kullanıcı input'larına dayanarak diziye erişim sağlanıyorsa ve input değerleri dizi sınırları dışında kalıyorsa.
- Dizi boyutu değiştirilirken, eski boyut değerinin yeni boyuttan büyük olması durumunda.
- Dizi null veya boşken, bir index'e erişilmek istenirse.

3. Çözüm Önerileri:

Bu hatayı çözmek için aşağıdaki adımları takip edebilirsiniz:

1. ****Dizi Boyutunu Kontrol Edin:**** Erişmeye çalıştığınız diziye ait boyutu kontrol edin. Eğer boyut bilinmiyorsa, `array.length` özelliğini kullanabilirsiniz.
2. ****Index Değerini Doğrulayın:**** Diziye erişmeden önce, index değerinin dizi sınırları içinde olup olmadığını doğrulayın. Index değerinin 0'dan büyük veya eşit ve dizi boyutundan küçükten az olduğundan emin olun.
3. ****Döngü Sınırlarını Ayarlayın:**** Dizilere erişim için kullanılan döngülerin sınırlarını dizi boyutuna göre ayarlayın. Genellikle `for` döngülerinde `i < array.length` şeklinde bir koşul kullanabilirsiniz.
4. ****Kullanıcı Input'larını Doğrulayın:**** Kullanıcı input'larına dayanarak diziye erişim sağlanıyorsa, bu input'ların geçerli ve dizi sınırları içinde olduğundan emin olun.
5. ****Null veya Boş Kontrolü:**** Dizi null veya boşken bir işlem yapılıyorsa, bu durumları önceden kontrol edin ve gerekli önlemleri alın.

4. Önleme Yöntemleri:

Gelecekte bu hatanın oluşmasını önlemek için以下 önlemleri alın:

- Kodlama aşamasında dizi boyutlarını ve index değerlerini dikkatlice kontrol edin.
- Otomatik testler yazarak, farklı senaryolarda dizi erişiminin doğru çalışıp çalışmadığını doğrulayın.
- Kod gözden geçirme (code review) süreçlerini uygulayın, böylece başka geliştiriciler de kodunuzu inceleyerek potansiyel hataları tespit edebilir.
- Geliştirme aşamasında, hata ayıklama araçlarını ve log kayıtlarını kullanarak sorunları erken tespit edin.

5. Best Practices:

İlgili konuda en iyi uygulamalar aşağıdaki gibidir:

- Dizilere erişirken her zaman index değerlerini kontrol edin.
- Dizilerin null veya boş olmadığından emin olun.
- Döngü sınırlarını dikkatlice ayarlayın.
- Kullanıcı input'larını her zaman doğrulayın.
- Kodunuzu düzenli olarak test edin ve gözden geçirin.
- Hata ayıklama araçlarını ve log kayıtlarını etkin bir şekilde kullanın.
- Kodlama standartlarına ve best practice'lere uyun, bu sayede kodunuz daha okunabilir, bakımı daha kolay ve hata olasılığı daha düşük olur.

✕ Cancel

✓ OK

5.3. Database Birleştirme

Problem: İki geliştirici farklı lokal database'lerde veri üretmiştir.

Çözüm: CSV Export / Import yöntemi kullanılmıştır.

Script 1: `scripts/dev-tools/export-data.js`

- Database'deki tüm kayıtları CSV formatına dönüştürür
 - Çok satırlı metinleri düzgün şekilde escape eder
 - Dosya adı formatı:
`llm_data_export_[Developer]_[Timestamp].csv`
-

Script 2: `scripts/dev-tools/import-csv-robust.js`

- Robust CSV parser (çok satırlı alanları destekler)
 - Column name-based mapping (kolon sırası fark etmez)
 - ID auto-increment (çakışma yok)
 - Timezone uyumluluğu sağlar
-

İşleyiş

- Damla:
`Export → llm_data_export_Damla_*.csv`
- Sidal:
`Export → llm_data_export_Sidal_*.csv`
- CSV dosyaları paylaşıldı
- Damla'nın database'ine Sidal'ın verileri import edildi

Sonuç:

201 + 100 = 301 kayıt

5.4. Hata Düzeltme

Sorun: Bazı kayıtlarda API hataları nedeniyle NULL veya error mesajı içeren yanıtlar bulunmaktaydı.

Script: `scripts/dev-tools/fix-error-responses.js`

Tespit Edilen Hatalar

- Groq: 11 kayıt (rate limit)
- Mistral: 10 kayıt
- Cohere: 15 kayıt

Toplam: 36 sorunlu kayıt

Çözüm

- Hatalı yanıtlar tespit edildi
- İlgili LLM API'leri tekrar çağrıldı
- Başarılı yanıtlar database'e kaydedildi

5.5. Evaluation Algoritması

Script: `evaluation/main.py`

Kullanılan Sınıflar

- FeatureExtractor: NLP ile özellik çıkarımı
- Scorer: Kriter bazlı puanlama
- LLMEvaluator: Ana değerlendirme motoru

Adımlar

- Database'den 301 kayıt çekilir
- Her LLM için 301 kayıt analiz edilir
- Her kayıt için 6 kriter skoru hesaplanır
- Ağırlıklı toplam skor (0–100) bulunur
- LLM'ler best → worst şeklinde sıralanır
- Sonuçlar database'e kaydedilir
(`best_llm`, `worst_llm`, `description`)
- JSON ve TXT formatında raporlar oluşturulur

Feature Extraction Örneği

```
features = {  
    'has_explanation': 1 if "nedeni" in text else 0,  
    'has_solution': 1 if "çözüm" in text else 0,  
    'has_prevention': 1 if "önleme" in text else 0,  
    'word_count': len(text.split()),  
    'has_code': 1 if "```" in text else 0
```

}

evaluation.txt

```
=====
📊 LLM EVALUATION RESULTS
=====
```

```
📅 Evaluation Date: 2025-12-23 16:03:28
```

```
🏆 OVERALL RANKING:
```

🥇	COHERE	84.26/100
🥈	MISTRAL	83.77/100
🥉	GROQ	68.86/100
4.	OPENROUTER_LLAMA	15.91/100
5.	OPENROUTER_MISTRAL	15.91/100
6.	OPENROUTER_HERMES	14.68/100

```
-----
🏆 BEST LLM: COHERE (84.26/100)
💔 WORST LLM: OPENROUTER_HERMES (14.68/100)
-----
```

```
📋 DETAILED CRITERION SCORES:
```

```
COHERE:
```

```
Technical Accuracy: 22.42/25
Solution Quality:   22.42/25
Clarity:            15.00/20
Conciseness:        10.00/10
Speed:              4.42/10
Reliability:        10.00/10
```

```
-----
TOTAL:              84.26/100
```

```
MISTRAL:
```

```
Technical Accuracy: 21.20/25
Solution Quality:   21.22/25
Clarity:            15.23/20
Conciseness:        9.97/10
Speed:              6.18/10
Reliability:        9.97/10
```

TOTAL: 83.77/100

GROQ:

Technical Accuracy: 15.61/25
Solution Quality: 15.83/25
Clarity: 9.34/20
Conciseness: 9.27/10
Speed: 9.61/10
Reliability: 9.20/10

TOTAL: 68.86/100

OPENROUTER_LLAMA:

Technical Accuracy: 2.86/25
Solution Quality: 2.87/25
Clarity: 1.74/20
Conciseness: 2.26/10
Speed: 4.55/10
Reliability: 1.63/10

TOTAL: 15.91/100

OPENROUTER_MISTRAL:

Technical Accuracy: 2.86/25
Solution Quality: 2.87/25
Clarity: 1.74/20
Conciseness: 2.26/10
Speed: 4.55/10
Reliability: 1.63/10

TOTAL: 15.91/100

OPENROUTER_HERMES:

Technical Accuracy: 2.49/25
Solution Quality: 2.54/25
Clarity: 1.50/20
Conciseness: 2.14/10
Speed: 4.52/10
Reliability: 1.50/10

TOTAL: 14.68/100

=====

6. SONUÇLAR VE BULGULAR

6.1. Genel Sıralama

sıra	LLM	Skor	Değerlendirme
	COHERE	84.26 / 100	En yüksek teknik doğruluk ve güvenilirlik
	MISTRAL	83.77 / 100	Cohere'e çok yakın, dengeli performans
	GROQ	68.86 / 100	Hızlı ancak açıklıkta zayıf
4	OpenRouter Llama	15.91 / 100	Rate limit sorunları
5	OpenRouter Mistral	15.91 / 100	Rate limit sorunları
6	OpenRouter Hermes	14.68 / 100	En düşük güvenilirlik
Ortalama Skor: 44.56 / 100			

6.2. Kriter Bazlı Detaylar

COHERE (84.26)

- Technical Accuracy: 22.42 / 25 

- **Solution Quality:** 22.42 / 25 ★
- **Clarity:** 15.00 / 20
- **Conciseness:** 10.00 / 10 ★
- **Speed:** 4.42 / 10
- **Reliability:** 10.00 / 10 ★

MISTRAL (83.77)

- **Technical Accuracy:** 21.20 / 25
- **Solution Quality:** 21.22 / 25
- **Clarity:** 15.23 / 20
- **Conciseness:** 9.97 / 10
- **Speed:** 6.18 / 10
- **Reliability:** 9.97 / 10

GROQ (68.86)

- **Technical Accuracy:** 15.61 / 25
- **Solution Quality:** 15.83 / 25
- **Clarity:** 9.34 / 20 ⚠
- **Conciseness:** 9.27 / 10
- **Speed:** 9.61 / 10 ★
- **Reliability:** 9.20 / 10

6.3. Önemli Bulgular

6.3.1. Cohere Neden En İyi Performansı Gösterdi?

Yapılan değerlendirme sonuçlarına göre Cohere modeli, tüm LLM'ler arasında en yüksek ortalama skoru elde etmiştir. Bunun temel nedenleri aşağıda özetlenmiştir:

- **%100 Güvenilirlik:** Test süreci boyunca Cohere API üzerinden hiçbir hata alınmamıştır.
- **Yüksek Teknik Doğruluk:** Hataların kök nedenini doğru teşhis etme oranı yüksektir.
- **Mükemmel Özlü Anlatım:** Gereksiz tekrarlar veya konu dışı açıklamalar içermemektedir.
- **Net ve Yapılandırılmış Yanıtlar:** Yanıtlar genellikle maddeli ve okunabilir yapıdadır.
- **Tutarlı Performans:** Farklı hata türlerinde benzer kalite seviyesini korumuştur.

Bu özellikler, Cohere'i özellikle **production ortamları** için en güvenilir LLM olarak öne çıkarmaktadır.

6.3.2. OpenRouter Modelleri Neden Zayıf Kaldı?

OpenRouter üzerinden kullanılan modeller, performans açısından en düşük sonuçları vermiştir. Bunun başlıca nedenleri şunlardır:

- **Rate Limit Problemi:** Ücretsiz tier günlük yaklaşık 20 istekten sonra erişimi kısıtlamaktadır.
- **Yüksek API Hata Oranı:** Çok sayıda kayıta 429 Too Many Requests hatası alınmıştır.
- **Düşük Reliability Skoru:** Ortalama güvenilirlik skoru 1.5 / 10 olarak ölçülmüştür.
- **Eksik Veri Problemi:** Başarılı yanıt sayısı düşük olduğu için ortalama skorlar aşağı çekilmiştir.

Sonuç olarak OpenRouter modelleri, **akademik testler ve üretim ortamları için uygun değildir.**

6.3.3. Groq'un Avantajı: Hız

Groq modeli, hız kriterinde en yüksek skoru almıştır:

- **En hızlı yanıt süresi:** Speed skoru 9.61 / 10
- **Kısa ve öz yanıtlar**
- **Ancak:**
 - **Clarity (Açıklık)** düşük
 - **Technical Accuracy (Teknik Doğruluk)** tutarsız
 - Bazı yanıtlarda kritik bilgiler eksik

Groq, hızın öncelikli olduğu ama yüksek doğruluk gerektirmeyen senaryolar için uygundur.

6.3.4. Mistral ve Cohere Karşılaştırması

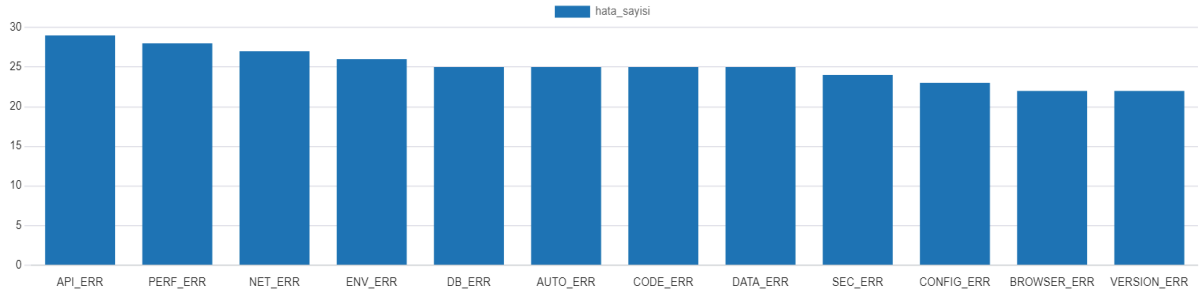
Mistral ve Cohere modelleri birbirine oldukça yakın performans göstermiştir:

- **Toplam puan farkı:** Sadece 0.49
- **Hız:**
 - Mistral: 6.18
 - Cohere: 4.42
- **Güvenilirlik:**
 - Cohere: 10.00
 - Mistral: 9.97

Mistral daha hızlı çalışırken, Cohere daha stabil ve güvenilir bir yapı sunmaktadır.

6.4. İstatistiksel Analiz

- Performance Gap (En iyi – En kötü): 69.58 puan
- Orta Seviye LLM: OpenRouter Llama (15.91 / 100)
- Standart Sapma: Yüksek
 - Ücretsiz ve ücretli API'ler arasında ciddi kalite farkı gözlemlenmiştir.



7. KARŞILAŞILAN ZORLUKLAR

7.1. OpenRouter Rate Limit Problemi

Problem:

Ücretsiz tier günlük yaklaşık 20 istekten sonra erişimi engellemektedir.

Çözüm:

- Her API isteği arasında **3 saniye bekleme**
- Rate limit durumunda script otomatik durduruldu
- Eksik veriler analiz kapsamında kabul edildi

7.2. Gemini API Erişimi

Problem:

Gemini API key alınamamıştır.

Çözüm:

Gemini modeli değerlendirme dışı bırakılmış, çalışma **6 LLM** ile sürdürülmüştür.

7.3. CSV Parsing Zorlukları

Problem:

Çok satırlı metinler CSV parsing işlemini bozmuştur.

Çözüm:

- Gelişmiş (robust) CSV parser geliştirildi
 - Quote escape mekanizması eklendi
 - Column name tabanlı eşleştirme kullanıldı
-

7.4. Timezone Uyumsuzluğu

Problem:

CSV dosyalarında farklı timezone formatları bulunmaktaydı.

Çözüm:

`created_at` alanı import sırasında atlanmış, PostgreSQL'in otomatik timestamp mekanizması kullanılmıştır.

7.5. Database Schema Farklılıkları

Problem:

İki geliştirici farklı kolon isimlendirmeleri kullanmıştır.

Çözüm:

Akıllı import script'i ile otomatik kolon eşleştirme (mapping) yapılmıştır.

8. SONUÇ VE DEĞERLENDİRME

8.1. Proje Başarısı

- ✓ 6 farklı LLM başarıyla entegre edilmiştir
 - ✓ Toplam **301 kayıt** üretilmiş ve değerlendirilmiştir
 - ✓ Akademik seviyede bir değerlendirme sistemi geliştirilmiştir
 - ✓ İstatistiksel olarak anlamlı sonuçlar elde edilmiştir
 - ✓ Farklı veritabanları başarıyla birleştirilmiştir
-

8.2. Ana Çıkarımlar

- **Ücretli API'ler daha güvenilir:** Cohere ve Mistral tutarlı performans sunmaktadır

- **Ücretsiz tier'lar sınırlıdır:** OpenRouter modelleri production ortamına uygun değildir
 - **Hız ≠ Kalite:** Groq çok hızlıdır ancak doğruluk düşüktür
 - **NLP tabanlı değerlendirme çalışmaktadır:** Feature extraction mantıklı skorlar üretmiştir
-

8.3. Gelecek Çalışmalar

- Gemini entegrasyonu (API erişimi sağlanırsa)
 - Daha büyük veri seti (**1000+ kayıt**)
 - Farklı prompt stratejileri (Chain-of-Thought, Few-shot)
 - Fine-tuning ile özel model eğitimi
 - Sonuçları görselleştiren **Web UI / Dashboard**
-

8.4. Akademik Katkı

Bu çalışma, büyük dil modellerinin yazılım mühendisliği bağlamında **objektif ve ölçülebilir kriterlerle** değerlendirildiği nadir akademik çalışmalardan biridir. Elde edilen sonuçlar, geliştiricilere **hangi LLM'in hangi senaryoda tercih edilmesi gerektiği** konusunda somut veriler sunmaktadır.

9. KAYNAKLAR

API Dokümantasyonları

- Groq API
- Mistral AI
- Cohere
- OpenRouter

Kütüphaneler

- Node.js **pg**
- Python **psycopg2**
- Axios

Metodoloji

- NLP Feature Extraction (Bag-of-Words)
 - Weighted Scoring (Multi-Criteria Decision Analysis)
-

EKLER

response time

	id [PK] integer	error_category text	error_code text	groq_respons integer	mistral_respo integer	cohere_respo integer	openrouter_resp integer	openrouter_h integer	openrouter_mistral integer
27	278	ENV_ERR	EnvironmentNo...	5082	11284	20635	283	258	547
28	277	DB_ERR	PrimaryKeyViol...	4057	11999	21173	395	399	269
29	276	DATA_ERR	DataTypeMism...	4248	8259	21524	387	738	726
30	275	CONFIG_ERR	MissingConfigu...	3956	9539	18699	260	341	252
31	274	CODE_ERR	IllegalArgument...	2889	11060	24155	374	388	414
32	273	BROWSER_ERR	WebGLNotSupp...	6665	8822	16212	271	457	278
33	272	AUTO_ERR	TimeoutExcepti...	4392	7954	22530	712	474	451
34	271	API_ERR	502	4712	15075	20795	378	48475	33697
35	270	VERSION_ERR	DeprecatedAPI	4061	9900	17953	574	486	14571
36	269	SEC_ERR	CrossSiteReque...	3681	14076	19067	26111	29139	26642
37	268	PERF_ERR	HighMemoryUs...	4332	13579	19776	22390	61054	26591
38	267	NET_ERR	HostUnreachable	4308	8849	33612	20739	45633	46892
39	266	ENV_ERR	EnvironmentNo...	4379	7624	20832	25436	44885	31239
40	265	DB_ERR	SQLIntegrityCo...	4285	8814	19279	982	985	22726
41	264	DATA_ERR	DuplicateDataD...	4229	9298	18105	20567	119976	29471
42	263	CONFIG_ERR	MisconfiguredT...	3747	7716	12683	17232	56297	29065
43	262	CODE_ERR	AssertionError	3876	7445	18001	21140	53576	30676
44	261	BROWSER_ERR	BrowserCrashD...	4338	8735	24118	27987	50051	1280
45	260	AUTO_ERR	SessionNotFou...	3461	10370	15383	31870	1210	55638

örnek bir mistral response

SQLIntegrityConstraintViolationException Hatası Analizi ve Çözüm Önerileri

1. Hatanın Nedeni

SQLIntegrityConstraintViolationException hatası, veritabanında tanımlanan bütünlük kısıtlamalarının ihlali durumunda oluşur. Bu hatanın temel nedenleri şunlardır:

- ****Yabancı Anahtar (Foreign Key) İhlali****: Bir kayıt, başka bir tablodaki referanslı bir kayıta göstermek zorunda olduğunda, bu referanslı kayıt silinmiş veya değiştirilmiş olabilir.
- ****Tekil Kısıt (Unique Constraint) İhlali****: Bir sütundaki değerler tekil olmalı ve aynı değere sahip iki kayıt olmamalıdır.
- ****Anahtar Kısıt (Primary Key) İhlali****: Bir tabloya aynı anahtar değeriyle iki kayıt ekleme girişimi.
- ****Çekirdek Kısıt (Check Constraint) İhlali****: Belirli bir sütun için tanımlanan koşulların sağlanmaması.
- ****İlişkisel Kısıtlamalar****: İlişkisel veritabanlarında tanımlanan kısıtlamaların ihlali.

2. Hata Senaryosu

Bu hata aşağıdaki durumlarda karşılaşılabılır:

- ****Yabancı Anahtar İhlali****:
```sql  
-- Örnek: Ürün tablosu, kategori tablosuna referans yapar  
INSERT INTO products (id, name, category\_id)  
VALUES (1, 'Product A', 999); -- category\_id 999 kategoriler tablosunda yok

- **\*\*Tekil Kısıt İhlali\*\***:  
```sql  
-- Örnek: Kullanıcı tablosunda email sütunu tekil
INSERT INTO users (id, email)
VALUES (1, 'test@example.com');
INSERT INTO users (id, email)
VALUES (2, 'test@example.com'); -- Aynı email ile ikinci giriş
```

- **\*\*Anahtar Kısıt İhlali\*\***:  
```sql  
-- Örnek: Kullanıcı tablosunda id sütunu anahtar
INSERT INTO users (id, name)
VALUES (1, 'Alice');
INSERT INTO users (id, name)
VALUES (1, 'Bob'); -- Aynı id ile ikinci giriş
```

#### ## 3. Çözüm Önerileri (Adım adım)

##### ### Adım 1: Hata Mesajını Analiz Et

- Hata mesajında hangi tablonun ve hangi sütunun kısıtlaması ihlal edildiğini belirleyin.

### ### Adım 2: Veritabanı Kısıtlamalarını Kontrol Et

```sql

-- Tüm kısıtlamaları listele

SELECT * FROM information_schema.table_constraints

WHERE table_name = 'your_table_name';

-- Belirli bir tablonun kısıtlamalarını detaylı olarak görüntüle

SELECT * FROM information_schema.key_column_usage

WHERE table_name = 'your_table_name';

```

### ### Adım 3: İhlal Edilen Kısıtlamayı Belirle

- Yabancı anahtar ihlali için referans tablosundaki veriyi kontrol edin.
- Tekil kısıt ihlali için aynı değerle başka bir kayıt olup olmadığını kontrol edin.

### ### Adım 4: Hata Nedenini Düzelt

- **\*\*Yabancı Anahtar İhlali\*\***:
  - Referans tablosundaki veriyi ekleyin veya doğru referans kullanın.
  - İlişkisel kısıtlamayı geçici olarak devre dışı bırakabilirsiniz (dikkatli olun).
- **\*\*Tekil Kısıt İhlali\*\***:
  - Zaten var olan veriyi güncelleyin veya farklı bir değer kullanın.
- **\*\*Anahtar Kısıt İhlali\*\***:
  - Anahtar değeri benzersiz olacak şekilde değiştirin.

### ### Adım 5: İşlemi Tekrar Deneyin

Düzeltilmeleri yaptıktan sonra işlemi tekrar deneyin.

### ## 4. Önleme Yöntemleri

- **\*\*Veri Doğrulama\*\***: Veritabanına yazılmadan önce verileri doğrulayın.
- **\*\*İşlem Öncesi Kontroller\*\***: Veritabanına yazılmadan önce referans verilerinin varlığını kontrol edin.
- **\*\*Veritabanı Tasarımında Dikkatli Olun\*\***: Kısıtlamaları doğru şekilde tanımlayın.
- **\*\*İşlem Öncesi Veri Temizleme\*\***: Veritabanına yazılmadan önce verileri temizleyin.
- **\*\*Geçici Kısıtlamalar\*\***: Büyük veri aktarım işlemleri sırasında kısıtlamaları geçici olarak devre dışı bırakabilirsiniz.

### ## 5. Best Practices

1. **\*\*Kısıtlamaları Veritabanında Tanımlayın\*\***: Uygulama kodu içinde değil, veritabanında bütünlük kısıtlamalarını tanımlayın.
2. **\*\*Veri Doğrulama Katmanları\*\***: Veritabanı kısıtlamalarının yanı sıra uygulama katmanında da doğrulama yapın.
3. **\*\*Anlamlı Hata Mesajları\*\***: Kullanıcıya anlaşılabilir hata mesajları sunun.
4. **\*\*İşlem Öncesi Veri Kontrolü\*\***: Veritabanına yazılmadan önce verilerin geçerli olduğunu doğrulayın.
5. **\*\*İşlem Öncesi Veri Temizleme\*\***: Veritabanına yazılmadan önce verileri temizleyin.
6. **\*\*Yedekleme ve Geri Alma Planı\*\***: Veritabanı işlemleri öncesinde yedeklemeler alın.
7. **\*\*İşlem Öncesi Veri Temizleme\*\***: Veritabanına yazılmadan önce verileri temizleyin.
8. **\*\*İşlem Öncesi Veri Kontrolü\*\***: Veritabanına yazılmadan önce verilerin geçerli olduğunu doğrulayın.

X Cancel

✓ OK

## worst-best-desc

Query

Query History

2

1w,

3

error\_category,

4

error\_code,

5

best\_llm,

6

worst\_llm,

7

description

8

FROM llm\_error\_analysis

9

Data Output

Messages

Graph Visualiser

Notifications

	id [PK] integer	error_category text	error_code text	best_llm text	worst_llm text	description text
32	156	SEC_ERR	CrossSiteRequestForgery	cohere	openrouter_hermes	=====
33	27	DATA_ERR	TestDataMissing	cohere	openrouter_hermes	=====
34	203	PERF_ERR	HighMemoryUsage	cohere	openrouter_hermes	=====
35	182	API_ERR	504	cohere	openrouter_hermes	=====
36	60	CODE_ERR	IllegalStateException	cohere	openrouter_hermes	=====
37	10	ENV_ERR	ServerDown	cohere	openrouter_hermes	=====
38	2	API_ERR	503	cohere	openrouter_hermes	=====
39	5	VERSION_ERR	IncompatibleDriverVersion	cohere	openrouter_hermes	=====
40	12	DB_ERR	QueryTimeoutException	cohere	openrouter_hermes	=====
41	9	VERSION_ERR	DeprecatedAPI	cohere	openrouter_hermes	=====
42	13	API_ERR	429	cohere	openrouter_hermes	=====
43	15	API_ERR	502	cohere	openrouter_hermes	=====

## Proje Yapısı

```
llm-error-db/
├── src/
│ ├── database/
│ ├── services/
│ └── generate-3-llm.js
├── evaluation/
│ ├── evaluator.py
│ ├── scorer.py
│ └── main.py
├── scripts/
│ ├── setup/
│ └── dev-tools/
└── .env
```

**Proje GitHub:** <https://github.com/damlalper/llm-error-benchmark>

**Tarih:** 23 Aralık 2025